

Tema 1. Introducción a Kotlin

Introducción a



- Es un lenguaje de programación moderno creado por JetBrains.
- Es el lenguaje **oficial para el desarrollo de apps Android** recomendado por Google.
- Se usa principalmente con Android Studio para crear aplicaciones en Android.
- Es **conciso, seguro y fácil de leer**.
- Es **100% interoperable con Java**.

Kotlin es básicamente la [evolución moderna de Java para Android](#): más corto, más seguro y más rápido de desarrollar.

Variables y tipos de datos

Variables: **var** y **val**

- **var**: variable **mutable**, puede cambiar de valor.
- **val**: variable **immutable**, no puede cambiar después de asignarla. (equivalente a **final** (constante))

Tipos de datos básicos

- **Int** → números enteros
- **Double** → números decimales
- **String** → texto
- **Boolean** → verdadero/falso

Char, Byte, Sort,Float, Double...

El ":" define tipo de datos

```
val edad: Int = 25  
val precio: Double = 99.99  
val nombre: String = "Ana"  
val esEstudiante: Boolean = true
```

¿Que pasa si no defino un tipo de datos?
por ejemplo: **val edad = 18**

Si los tipos datos no son definidos de forma
explicita, **Kotlin los define
automáticamente.**

Null Safety

Null safety (?)

- Kotlin evita errores de **referencia nula**.
- El operador **?** permite que una variable sea nula.
- **Evita** el problema **NullPointerException** de Java

```
var nombre: String? = null  
println(nombre?.length) //  imprime null en vez de error
```

Null Safety

Null safety (!!)

- Kotlin permite manejar `NullPointerException` si se desea
- El operador `!!` indica que un valor **no debe ser null** o de lo contrario **generará error**
- **El funcionamiento** sería el mismo que el `NullPointerException` de Java

```
numero = readLine()!!.toInt()
```

En este ejemplo, lee un número del teclado para convertirlo en entero, si no se introduce nada(null) salta un error

Variables y tipos de datos

Tipos de conversión

Para convertir un tipo de dato a otro compatible, por ejemplo convertir un Int a un Short, debe usar la funciones de conversión.

- `toByte()`
- `toShort()`
- `toInt()`
- `toLong()`
- `toFloat()`
- `toDouble()`
- `toChar()`

ToString() -> no siempre es necesario gracias a la interpolación

```
fun main(args: Array<String>)  
{  
    var x:Int = 55  
    var y:Short = x.toShort()  
  
    println("X = $x")  
    println("Y = $y")  
}
```

```
val numero = 25  
println("El número es $numero")
```

Estructuras de control

Estructuras de control

- **if** — Condicional simple

```
val edad = 18
if (edad >= 18) {
    println("Adulto")
} else {
    println("Menor")
}
```

- **when** — Condicional múltiple (switch avanzado)

```
val dia = 3
when(dia) {
    1 -> println("Lunes")
    2 -> println("Martes")
    else -> println("Otro día")
}
```

Estructuras de control

Estructura de selección **When**, ventajas frente al **Switch** de Java

```
int dia = 3;
switch(dia) {
    case 1:
        System.out.println("Lunes");
        break;
    case 2:
        System.out.println("Martes");
        break;
    default:
        System.out.println("Otro día");
        break;
}
```



```
val dia = 3
when(dia) {
    1 -> println("Lunes")
    2 -> println("Martes")
    else -> println("Otro día")
}
```

1º: Es mucho más **limpio** y **legible**

2º: Cada caso es una **expresión completa** sin necesidad de {}

Estructuras de control

Estructura de selección **When**, ventajas frente al **Switch** de Java

```
val dia = 5
when(dia) {
    1, 2, 3 -> println("Principio de semana")
    4, 5 -> println("Casi fin de semana")
    else -> println("Fin de semana")
}
```

```
val edad = 20
when {
    edad < 13 -> println("Niño")
    edad in 13..19 -> println("Adolescente")
    else -> println("Adulto")
}
```

3º: **Múltiples valores** en un mismo caso

4º: **Condiciones complejas** como rangos y condiciones booleanas

```
val resultado = when(dia) {
    1 -> "Lunes"
    2 -> "Martes"
    else -> "Otro día"
}
println(resultado)
```

5º: Puede usarse como una expresión que **devuelve un valor**

Estructuras de control

Ejercicio:

Realiza en [Java y Kotlin](#) una función que convierta las notas numéricas en su valor: insuficiente (0-5) suficiente(5-6), notable(6-8) y Sobresaliente(9-10)

Realiza en [Java y Kotlin](#) una función que tenga un entero que es “edad” y un booleano que es “estudiante”, con esos datos debe dar las siguientes salidas:

- Edad menor de 10 años y estudiante=false: “niño pequeño, no puede trabajar”
- Edad menor de 10 años y estudiante=true: “niño pequeño”
- Edad entre 10 y 16 años y estudiante=false: “sin edad de trabajar”
- Edad entre 10 y 16 años y estudiante=true: “estudiante juvenil”
- Edad mayor de 16 años y estudiante=true: “Estudiante”
- Edad mayor de 16 años y estudiante=false: “Trabajador”

Compilador online de Kotlin: <https://play.kotlinlang.org/>

Estructuras de control

```
fun main() {  
  
    var edad= 20  
    var estudiante = false  
  
    var resultado = when {  
        edad < 10 && !estudiante -> "Niño pequeño, no puede trabajar"  
        edad < 10 && estudiante -> "Niño pequeño"  
        edad in 10..16 && !estudiante -> "Sin edad de trabajar"  
        edad in 10..16 && estudiante -> "Estudiante juvenil"  
        edad > 16 && estudiante -> "Estudiante"  
        edad > 16 && !estudiante -> "Trabajador"  
        else -> "Datos inválidos"  
    }  
    println(resultado)  
}
```

Estructuras de control

El bucle **for** se usa para **recorrer rangos, colecciones o arrays** no existe el for clásico de Java

Ejemplo con rango

```
<> Kotlin

for (i in 1..5) {
    println(i)
}
```

Salida:

1 2 3 4 5

Con paso (step):

```
<> Kotlin

for (i in 1..10 step 2) {
    println(i)
}
```

Cuenta regresiva (downTo):

```
<> Kotlin

for (i in 10 downTo 1) {
    println(i)
}
```

Ejemplo recorriendo una lista

```
<> Kotlin

val nombres = listOf("Ana", "Luis", "Carlos")

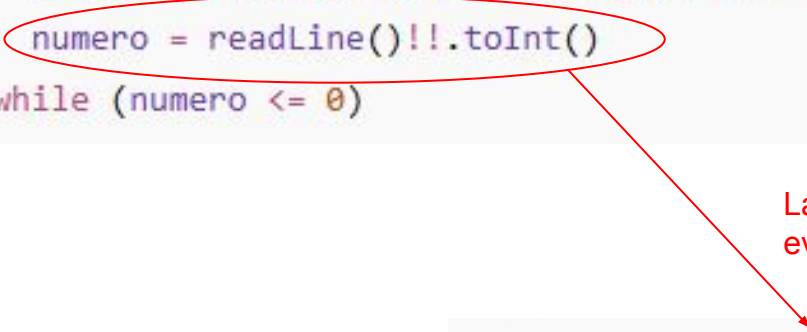
for (nombre in nombres) {
    println(nombre)
}
```

Estructuras de control

El bucle **do while** ejecuta el bloque **al menos una vez**, porque la condición se evalúa al final. Muy útil para validación de datos.

```
var numero: Int

do {
    println("Introduce un número mayor que 0")
    numero = readLine()!!.toInt()
} while (numero <= 0)
```



La mejor forma de hacer esto para evitar error de null o de formato es:

```
val numero = readLine()?.toIntOrNull()
```

Arrays y Listas

Los arrays en Kotlin son muy similares a java, se definen con la palabra clave `arrayOf`

```
val nombres = arrayOf("Ana", "Luis", "Carlos")
```

```
println(nombres[0]) // Ana
println(nombres[1]) // Luis
```

```
val numeros = arrayOf(1, 2, 3, 4, 5)
```

Recorrer Array con `for`

```
val numeros = arrayOf(10, 20, 30)
```

```
for (n in numeros) {
    println(n)
}
```

'n' coge el valor del elemento

Recorrer Array con `indice`

```
for (i in numeros.indices) {
    println("Posición $i = ${numeros[i]}")
}
```

'i' coge el valor del índice
(posición del elemento)

Tamaño del Array

```
println(numeros.size)
```

```
val numeros = intArrayOf(1, 2, 3)
val decimales = doubleArrayOf(1.5, 2.5)
val booleanos = booleanArrayOf(true, false)
```

Arrays y Listas

Las listas se definen con `listOf` se diferencian de los arrays básicamente en el tamaño y mutabilidad:

Característica	Array	List
Creación	<code>arrayOf(1, 2, 3)</code>	<code>listOf(1, 2, 3)</code>
Tamaño	Fijo (puedes cambiar elementos, pero no el tamaño)	Fijo si es <code>listOf</code> , mutable si <code>mutableListOf</code>
Mutabilidad	Los elementos se pueden cambiar (<code>arr[0] = 10</code>)	<code>listOf</code> → no permite cambiar elementos, <code>mutableListOf</code> → sí
Tipos específicos	<code>intArrayOf</code> , <code>doubleArrayOf</code> , etc. (más eficiente)	Todos los tipos → no hay optimización de primitivos
Funciones útiles	<code>size</code> , <code>get</code> , <code>set</code>	<code>size</code> , <code>get</code> , <code>contains</code> , <code>indexOf</code> , <code>forEach</code> , etc.

Funciones

Funciones (**fun**)

- Bloques de código reutilizables.
- Pueden recibir **parámetros** y devolver **valores**.

Recibe el String nombre y retorna un string

```
fun saludar(nombre: String): String {  
    return "Hola, $nombre!"  
}  
  
println(saludar("Ana")) // Imprime: Hola, Ana!
```

Fijaros en el siguiente código: ¿Que llama la atención? ¿Os recuerda a algo?

```
val nombre = "Ana"  
val edad = 25  
  
println("Hola, $nombre. Tienes $edad años.")
```

Funciones

Interpolación de variables: Al igual que en C, en Kotlin se pueden insertar directamente variables dentro de un texto usando el operador \$

```
val nombre = "Ana"
val edad = 25

println("Hola, $nombre. Tienes $edad años.")
```

Funciones lambda:

Recibe dos enteros y retorna un entero

```
val maximo: (Int, Int) -> Int = { a, b -> if (a > b) a else b }

fun main() {
    println(maximo(10, 20)) // 20
    println(maximo(50, 25)) // 50
}
```

Expresión

Funciones

Ejercicio: Crea las siguientes funciones con y sin lambda:

1. Calcular el doble de un número
2. Sumar dos números
3. Recibe dos String con un nombre y un saludo y devuelve un mensaje personalizado
4. Obtener los números pares de una lista

Objetos

Similar que en java:

```
class Persona(val nombre: String, var edad: Int) {  
    fun saludar() {  
        println("Hola, me llamo $nombre y tengo $edad años")  
    }  
}  
  
fun main() {  
    val ana = Persona("Ana", 20)  
    ana.saludar() // Hola, me llamo Ana y tengo 20 años  
}
```

Objetos

```
class Vehiculo(  
    val marca: String,  
    val modelo: String,  
    val año: Int,  
    val color: String,  
    val kilometraje: Int,  
    // ... más atributos hasta 120  
    val atributo120: String  
) {  
    // Función 1: mostrar información básica  
    fun mostrarInfo() {  
        println("Marca: $marca, Modelo: $modelo, Año: $año")  
    }  
}
```

```
fun main() {  
    val auto = Vehiculo(  
        marca = "Toyota",  
        modelo = "Corolla",  
        año = 2020,  
        color = "Rojo",  
        kilometraje = 30000,  
        atributo120 = "valor"  
    )  
  
    auto.mostrarInfo()  
}
```